React – TPS PRATIQUES

Exercices Formulaires et Notes avec Style

Partie 1 : Formulaires utilisateur et mot de passe

1. Créez un formulaire pour saisir **Nom, Prénom, Email, Mot de passe**.
   - Affichez **en direct** toutes les données saisies sous le formulaire.
2. Ajoutez une validation simple pour l'email : il doit contenir le caractère @.
   - Affichez un **message d'erreur** si l'email est invalide.
3. Ajoutez un champ **Confirmation mot de passe**.
   - Affichez une **alerte** si le mot de passe et la confirmation ne correspondent pas.
4. Ajoutez un champ **Super mot de passe**.
   - Le mot de passe doit contenir **au moins 8 caractères**.
   - Affichez un message d'erreur si ce n'est pas respecté.

Partie 2 : Gestion des notes d'étudiant

5. Créez un formulaire pour ajouter des notes :

- <select> pour choisir la matière (ex. informatique, math, anglais, arabe)
- <input> pour saisir la note
- Bouton **Ajouter** pour enregistrer la note

6. Affichez la liste des notes ajoutées avec un **bouton** ❌ à côté de chaque note pour la supprimer.
7. Calculez et affichez la **moyenne des notes**.

Formateur : AARAB HAFID

8. Affichez un **message d'erreur** si l'utilisateur saisit une note vide ou non numérique.

Partie 3 : Application du style

.Pour tous les formulaires et affichages ci-dessus :

- Appliquez un **style CSS ou inline** pour que les champs <input> et <select> soient lisibles et bien espacés.
- Stylisez les **boutons** "Ajouter" ou "Vérifier" pour qu'ils soient visibles et facilement cliquables.
- Stylisez les **messages d'erreur** pour qu'ils ressortent visuellement (ex. couleur rouge, italique).

Partie 2 : Listes et calculs dynamiques

Créer un composant React qui calcule automatiquement le prix total de location d'une voiture en fonction du type choisi et du nombre de jours.

Instructions :

1. Créez un composant LocationVoiture.jsx.
2. Ajoutez un <select> permettant de choisir le type de voiture :
   - Basic → 100 MAD/jour
   - Extra → 250 MAD/jour
   - Luxe → 400 MAD/jour
3. Ajoutez un <input> pour que l'utilisateur saisisse le nombre de jours de location.
4. Affichez **en direct** le prix total : (tarif × nombre de jours) dès que l'utilisateur change le type de voiture ou le nombre de jours.

5. **Style** : appliquez un style clair pour le <select>, <input> et le résultat du prix (couleur, bordure, padding…).

Exercice : Panier de fruits

Créer un composant React permettant à l'utilisateur d'ajouter des fruits dans une liste et de les afficher immédiatement.

Instructions :

1. Créez un composant PanierFruits.jsx.
2. Ajoutez un <input> pour saisir le nom du fruit.
3. Ajoutez un bouton Ajouter qui ajoute le fruit à la liste lorsqu'on clique dessus.
4. Affichez tous les fruits ajoutés dans une liste <ul><li> en temps réel.
5. Style : appliquez un petit espace entre chaque fruit et un fond léger pour chaque item de la liste pour

. Supprimer un fruit

- Ajoutez un **bouton** ❌ à côté de chaque fruit pour le supprimer de la liste.
- **Question style :** bouton visible avec couleur rouge pour le supprimer.

. Ajouter état "favori"

- Chaque fruit peut être marqué comme **favori** en cliquant sur une **étoile** ⭐ .
- Les favoris doivent changer visuellement (ex. étoile pleine ou couleur différente).

. Afficher seulement les fruits favoris

- Créez un paragraphe séparé pour **afficher uniquement les fruits favoris**.
- **Question style :** utilisez italique ou couleur différente pour distinguer les favoris.

Filtrer les fruits

- Ajoutez un **champ de recherche** pour filtrer les fruits affichés (**affichage seulement**, sans modifier la liste originale).
- **Question style :** input bien lisible, avec placeholder clair.

Ajouter quantité pour chaque fruit

- Chaque fruit a maintenant une **quantité** (par défaut 1).
- Ajoutez deux boutons + et − pour **augmenter ou diminuer la quantité** (minimum 1, maximum 20).
- **Question style :** boutons visibles et distincts, quantité centrée.

Exercice : Compteur simple avec sous-composant

Créer un compteur React interactif et afficher une valeur dérivée dans un sous-composant.

Instructions :

1. Créez un composant Compteur.jsx avec :
   - Un bouton + pour augmenter le compteur.
   - Un bouton - pour diminuer le compteur.
   - Un affichage de la valeur actuelle du compteur.
2. Créez un sous-composant DoubleCompteur.jsx qui reçoit la valeur du compteur en props et affiche le double de cette valeur.
3. Style :
   - Ajoutez une couleur de fond ou une bordure autour du sous-composant pour qu'il se distingue clairement du compteur principal.

- Les boutons doivent être visibles et facilement cliquables.

17. Changer couleur du compteur

- La couleur du compteur doit changer selon la valeur :
  - Vert si >0
  - Rouge si <0
  - Noir si =0
- Question style : appliquez style inline ou CSS pour la couleur.

. Message conditionnel si compteur =0

- Affichez un message "Compteur à zéro" seulement lorsque la valeur du compteur est 0.
- **Question style :** message en gras ou italique pour être bien visible.

Partie 3 : Consommation d'API

Exercice 19 – Liste des clients

- **Objectif :** Récupérer la liste des clients depuis l'API locale http://localhost:3001/clients et l'afficher dans un tableau <table>.
- **Instructions :**
  1. Utilisez fetch pour récupérer les clients au montage du composant.
  2. Affichez les colonnes **Nom** et **Email**.
- **Style :** Ajoutez une bordure au tableau et une couleur de fond légère pour les en-têtes.

Exercice 20 – Ajouter un client

- **Objectif :** Créer un formulaire pour ajouter un nouveau client et mettre à jour la liste immédiatement.
- **Instructions :**
    1. Champs du formulaire : Nom et Email.
    2. Au clic sur **Ajouter**, poster le client à l'API et mettre à jour la liste affichée.
- **Style :** Bouton **Ajouter** visible et cliquable, champs avec marge et padding.

Exercice 21 – Supprimer un client

- **Objectif :** Ajouter un bouton pour supprimer un client depuis la liste et mettre à jour l'affichage.
- **Instructions :**
    1. Chaque ligne du tableau doit avoir un bouton **Supprimer**.
    2. Au clic, supprimer le client via l'API et retirer la ligne du tableau.
- **Style :** Bouton **Supprimer** coloré (rouge) et bien visible.

Exercice 22 – Message de chargement

- **Objectif :** Afficher un message **"Chargement..."** pendant que les données sont récupérées depuis l'API.
- **Instructions :**
    1. Affichez le message tant que la réponse de l'API n'est pas reçue.
- **Style :** Texte en italique et centré pour signaler le chargement.

Exercice 23 – Liste des produits

- **Objectif :** Récupérer et afficher la liste des produits depuis http://localhost:3001/products.
- **Instructions :**
    1. Chaque produit doit afficher **nom, quantité** et une **image** (cube.png).
- **Style :** Ajoutez un fond léger pour chaque item et un peu de marge entre les produits.

Partie 4 : Routage / Navigation

Exercice 24 – Menu de navigation

- **Objectif :** Créer un menu avec 3 liens (**Accueil**, **Clients**, **Produits**) et afficher le composant correspondant à chaque lien.
- **Instructions :**
  1. Utilisez React Router pour gérer la navigation.
  2. Chaque lien doit ouvrir le composant correspondant dans la zone principale de la page.
- **Style :**
  - Les liens doivent être clairement visibles et espacés.
  - Ajoutez un fond léger ou surbrillance au lien actif.

Exercice 25 – Sous-route client

- **Objectif :** Créer une sous-route /clients/:id pour afficher le détail d'un client.
- **Instructions :**
  1. Lorsqu'on clique sur un client dans la liste, naviguer vers /clients/ID.
  2. Afficher le **Nom** et **Email** du client dans le composant détail.
- **Style :**
  - Encadrer les informations du client avec une bordure ou un fond différent pour bien distinguer le détail.

Partie 5 : HOOKS –

Exercice 1 – Focus sur input (5 pts)

- **Objectif :** Créer un input "Nom" et un bouton "Focus".
- **Instructions :**

- Au clic sur le bouton, l'input doit recevoir le focus.
- Utiliser useRef pour accéder directement à l'élément DOM.

- **Style :** Ajouter un contour visible à l'input quand il est focus.

Exercice 2 – Compteur automatique (5 pts)

- **Objectif :** Créer un compteur qui s'incrémente automatiquement toutes les secondes.
- **Instructions :**
  - Afficher le nombre de secondes écoulées.
  - Ajouter un bouton "Stop" pour arrêter le compteur.
  - Stocker l'ID du timer avec useRef.
- **Style :** Texte du compteur en gras et couleur différente.

Exercice 3 – Factorielle avec useMemo (5 pts)

- **Objectif :** Créer un input pour saisir un nombre et calculer sa factorielle.
- **Instructions :**
  - Afficher la factorielle.
  - Utiliser useMemo pour recalculer uniquement lorsque le nombre change.
- **Style :** Encadrer le résultat dans un paragraphe avec fond clair.

Exercice 4 – Liste avec useCallback (5 pts)

- **Objectif :** Créer une liste de nombres et un bouton "Ajouter un nombre aléatoire".
- **Instructions :**
  - Créer un composant enfant Liste.jsx qui reçoit la fonction d'ajout via props.
  - Utiliser useCallback pour que la fonction ne soit pas recréée à chaque rendu.
- **Style :** Chaque élément de la liste avec un fond léger et marge.

Exercice 5 – Compteur avec useReducer (5 pts)

- **Objectif :** Créer un compteur avec boutons +, -, Reset.
- **Instructions :**
  - Utiliser useReducer pour gérer le state du compteur.
- **Style :** Boutons colorés et espacés.

Exercice 6 – Mini Todo list (5 pts)

- **Objectif :** Ajouter et supprimer des tâches.
- **Instructions :**
  - Utiliser useReducer pour gérer le state des tâches.
- **Style :** Liste avec marge, chaque tâche encadrée, bouton ❌ rouge pour supprimer.

Exercice 7 – Thème clair/sombre avec useContext (5 pts)

- **Objectif :** Créer un contexte ThemeContext pour stocker le thème clair/sombre.
- **Instructions :**
  - Composant parent fournit le thème.
  - Composant enfant applique un style différent selon le thème via useContext.
- **Style :** Fond sombre pour thème sombre, fond clair pour thème clair, texte lisible.

Exercice 8 – Compteur avec contexte StepContext (5 pts)

- **Objectif :** Créer un composant compteur qui lit un contexte StepContext.
- **Instructions :**
  - Le contexte fournit la valeur d'incrément/décrément.
  - Utiliser useContext pour modifier le compteur selon le step.
- **Style :** Compteur avec bordure et couleur selon valeur.

Exercice 9 – Liste de fruits avec useReducer + useCallback (5 pts)

- **Objectif :** Créer un input pour ajouter des fruits dans une liste.
- **Instructions :**
  - Utiliser useReducer pour gérer la liste.
  - Utiliser useCallback pour la fonction d'ajout afin de ne pas la recréer à chaque rendu.
- **Style :** Chaque fruit avec fond léger, espace entre items, bouton ❌ pour supprimer.

Exercice 10 – Mini dashboard (10 pts)

- **Objectif :** Intégrer plusieurs hooks dans un seul composant :
  - useRef pour focus sur un input
  - useReducer pour gérer une liste d'éléments
  - useMemo pour calculer le nombre total d'éléments
  - useCallback pour ajouter un élément à la liste
  - useContext pour appliquer un thème clair/sombre
- **Style :**
  - Encadrer la liste avec bordure, fond clair.
  - Boutons colorés et espacés.
  - Résultat total en gras et couleur.